Estudiantes: David Maximiliano Pablo Cuy Carné: 261341
José David Ixcot Estrada Carné: 261980
Curso: Programación Orientada a Objetos
Docente: Erick Francisco Marroquín Rodríguez

# Análisis del Laboratorio 4

## 1. Requisitos funcionales

**Información que guarda el sistema**

- Equipos: su código (que no se puede repetir), su nombre, la empresa a la que representa y los puntos de tabla que lleva. Los puntos de tabla no son los goles ni los puntos que se anotan en un partido.
- Encuentros: el deporte, los dos equipos, el marcador de cada uno, su estado (sin programar, programado o finalizado) y, cuando se programa, la fecha, la hora y la sede.
- Exhibiciones: el título, su estado (sin programar o programada) y, cuando se programa, la fecha, la hora y la sede. No tienen equipos ni marcador.

**Lo que el sistema permite hacer**

- Registrar equipos.
- Crear encuentros, eligiendo el deporte y los dos equipos, y crear exhibiciones.
- Programar y reprogramar cualquier evento.
- Registrar el resultado de un encuentro.
- Ver la tabla de posiciones: por puntos de mayor a menor y, si dos equipos empatan, por nombre.
- Ver el calendario: solo los eventos que ya tienen fecha, ordenados por fecha, hora y sede.

**Cuándo es válida cada operación**

- Registrar un equipo: el código, el nombre y la empresa no pueden ir vacíos, y el código no puede estar repetido.
- Crear un encuentro: los dos equipos tienen que existir y no pueden ser el mismo. Crear una exhibición: tiene que tener título.
- Programar o reprogramar: la hora va de 0 a 23 y la sede no puede ir vacía. La sede no puede tener otro evento a esa misma fecha y hora y, si es un encuentro, ninguno de sus equipos puede estar jugando otro partido a esa hora. Un encuentro que ya terminó no se puede reprogramar.
- Registrar un resultado: el encuentro tiene que estar programado y sin resultado. Los marcadores no pueden ser negativos y en baloncesto no puede haber empate.
- Si algo no se cumple, el programa muestra un mensaje y no cambia nada. Si se escribe algo con el formato equivocado, como letras donde va un número, lo vuelve a pedir sin cerrarse.

## 2. Reglas del deporte y comportamiento compartido

Algunas reglas cambian según el deporte y otras son iguales para todos. En el cuadro está cada una, quién la resuelve y por qué.

| # | Regla o comportamiento | Quién lo resuelve | Por qué |
|---|---|---|---|
| 1 | Si el marcador es válido: en fútbol sala se puede empatar y en baloncesto no | FutbolSala y Baloncesto (clases concretas), con esMarcadorValido | Cada deporte tiene su propia regla. Encuentro la deja pendiente (método abstracto) y cada deporte la completa. |
| 2 | Cuántos puntos de tabla gana cada equipo: fútbol sala 3, 1 o 0; baloncesto 2 o 1 | FutbolSala y Baloncesto (clases concretas), con calcularPuntosTabla | También cambia con el deporte. Así no hace falta un if que pregunte qué deporte es. |
| 3 | El nombre del deporte | FutbolSala y Baloncesto (clases concretas), con getDeporte | Solo cada deporte sabe cómo se llama. |
| 4 | Tener dos equipos, un marcador, un estado y un horario | Encuentro (clase abstracta) | Todos los encuentros lo tienen, sin importar el deporte, así que se escribe una sola vez. |
| 5 | Los pasos para registrar un resultado | Encuentro (clase abstracta), con registrarResultado | Los pasos son iguales en todos los deportes: revisar, guardar el marcador, cerrar el encuentro y sumar los puntos. Solo le pregunta al deporte si el marcador vale y cuántos puntos da. |
| 6 | Programarse y aparecer en el calendario | Interfaz Programable (la cumplen Encuentro y Exhibicion) | Encuentros y exhibiciones hacen esto, pero no se parecen en nada más, así que ninguno debe heredar del otro. |
| 7 | El orden del calendario: fecha, hora y sede | Clase Horario, con compareTo (Programable lo usa) | El orden se escribe una sola vez y sirve igual para encuentros y exhibiciones. |
| 8 | Si todavía se puede reprogramar | Cada clase, con puedeProgramarse de Programable | Un encuentro que ya terminó no se mueve; una exhibición siempre se puede mover. |
| 9 | Recibir un resultado y repartir puntos de tabla | Interfaz Puntuable (solo la cumple Encuentro) | Solo los encuentros dan puntos; las exhibiciones no. |
| 10 | Revisar que la sede o un equipo no estén ocupados | Copa (clase concreta) | Para saberlo hay que ver todos los eventos juntos, y la copa es la que los guarda. |
| 11 | El orden de la tabla: puntos y luego nombre | Equipo (clase concreta), con compareTo | Solo depende de los puntos y del nombre del equipo. |

## 3. Estados de un encuentro y de una exhibición

Un encuentro pasa por tres estados: sin programar, programado y finalizado. Una exhibición solo pasa por los dos primeros, porque nunca recibe resultado. Si una operación no se puede hacer en el estado en que está el evento, se rechaza y el evento se queda igual.

| # | Evento | Estado | Operación | Pasa a | Qué se necesita |
|---|---|---|---|---|---|
| 1 | Encuentro | — | Crear encuentro | Sin programar | Dos equipos distintos. |
| 2 | Encuentro | Sin programar | Programar | Programado | Hora de 0 a 23 y que ni la sede ni los equipos estén ocupados a esa hora. |
| 3 | Encuentro | Programado | Reprogramar | Programado | Lo mismo que al programar. |
| 4 | Encuentro | Programado | Registrar resultado | Finalizado | Marcador válido para el deporte. Aquí se suman los puntos de tabla. |
| 5 | Encuentro | Sin programar | Registrar resultado | Sigue sin programar | Se rechaza: primero hay que programarlo. |
| 6 | Encuentro | Finalizado | Programar o registrar resultado | Sigue finalizado | Se rechaza: el encuentro ya terminó. |
| 7 | Exhibición | — | Crear exhibición | Sin programar | Que tenga título. |
| 8 | Exhibición | Sin programar | Programar | Programada | Hora de 0 a 23 y sede libre a esa hora. |
| 9 | Exhibición | Programada | Reprogramar | Programada | Lo mismo que al programar. |

## 4. Clases y responsabilidades

### Clases

| Clase | Tipo | De qué se encarga |
|---|---|---|
| Programable | Interfaz | Todo lo que va al calendario: programarse, dar su fecha, hora y sede, y ordenarse. |
| Puntuable | Interfaz | Todo lo que recibe un resultado y reparte puntos de tabla. |
| Encuentro | Clase abstracta | Lo común de todos los encuentros: dos equipos, marcador, estado y horario. Cumple las dos interfaces. |
| FutbolSala | Clase concreta | Las reglas del fútbol sala. |
| Baloncesto | Clase concreta | Las reglas del baloncesto. |
| Exhibicion | Clase concreta | Clínicas y demostraciones. Solo cumple Programable. |
| Horario | Clase concreta | La fecha, hora y sede de un evento, y el orden del calendario. |
| Equipo | Clase concreta | Los datos del equipo y sus puntos. Se compara con otros equipos para ordenar la tabla. |
| Copa | Clase concreta | Guarda los equipos y los eventos, revisa que cada operación sea válida y se puede recorrer por su calendario. |
| EstadoEvento | Enum | Los estados posibles: sin programar, programado y finalizado. |
| Main | Clase concreta | El menú, los datos iniciales y el método main. |

### Propiedades

| # | tipoDato | nombre | visibilidad | motivo | Clase |
|---|---|---|---|---|---|
| 1 | EstadoEvento | SIN_PROGRAMAR, PROGRAMADO, FINALIZADO | public static final | Constantes del enum; la exhibición solo usa las dos primeras. | EstadoEvento |
| 2 | String | texto | private final | Texto para mostrar el estado ("Sin programar"). | EstadoEvento |
| 3 | String | codigo | private | Identifica al equipo; no se repite ni va vacío. Se guarda en mayúsculas. | Equipo |
| 4 | String | nombre | private | Nombre del equipo; se usa para desempatar la tabla. | Equipo |
| 5 | String | empresa | private | Empresa a la que representa. | Equipo |
| 6 | int | puntosTabla | private | Puntos de tabla acumulados; no son los goles ni los puntos anotados. | Equipo |
| 7 | LocalDate | fecha | private | Día del evento. | Horario |
| 8 | int | hora | private | Hora de inicio, de 0 a 23. | Horario |
| 9 | String | sede | private | Lugar del evento; no puede ir vacía. | Horario |
| 10 | Equipo[] | equipos | private | Arreglo de 2: local (0) y visitante (1). | Encuentro |
| 11 | int[] | marcador | private | Marcador del local y del visitante; enteros no negativos. | Encuentro |
| 12 | EstadoEvento | estado | private | Sin programar, Programado o Finalizado; solo cambia con programar y registrarResultado. | Encuentro |
| 13 | Horario | horario | private | Fecha, hora y sede; null mientras no esté programado. | Encuentro |
| 14 | int | PUNTOS_VICTORIA | private static final | Constante 3. | FutbolSala |
| 15 | int | PUNTOS_EMPATE | private static final | Constante 1. | FutbolSala |
| 16 | int | PUNTOS_DERROTA | private static final | Constante 0. | FutbolSala |

| | | | | | |
|---|---|---|---|---|---|
| 17 | int | PUNTOS_VICTORIA | private static final | Constante 2. | Baloncesto |
| 18 | int | PUNTOS_DERROTA | private static final | Constante 1. | Baloncesto |
| 19 | String | titulo | private | Título de la exhibición ("Clínica de baloncesto"). | Exhibicion |
| 20 | EstadoEvento | estado | private | Sin programar o Programado. | Exhibicion |
| 21 | Horario | horario | private | Fecha, hora y sede; null mientras no esté programada. | Exhibicion |
| 22 | ArrayList<Equipo> | equipos | private | Todos los equipos de la copa. | Copa |
| 23 | ArrayList< Programable> | eventos | private | Todos los eventos (encuentros y exhibiciones) con el tipo de la interfaz del calendario. | Copa |
| 24 | ArrayList< Puntuable> | encuentros | private | Los mismos encuentros vistos como Puntuable; guarda referencias, no copia datos. | Copa |
| 25 | Scanner | scanner | private static | Lector único de la consola. | Main |
| 26 | Copa | copa | private static | Copa que se administra durante la ejecución. | Main |

## Métodos

| # | retorno | nombre | visibilidad | motivo | Clase |
|---|---|---|---|---|---|
| 1 | void | programar(Horario horario) | public abstract | Contrato: asigna o cambia la fecha, hora y sede. | Programable |
| 2 | boolean | puedeProgramarse() | public abstract | Contrato: indica si todavía se puede programar o reprogramar. | Programable |
| 3 | boolean | estaProgramado() | public abstract | Contrato: indica si ya tiene horario. | Programable |
| 4 | Horario | getHorario() | public abstract | Contrato: devuelve el horario o null. | Programable |
| 5 | ArrayList<Equipo> | getParticipantes() | public abstract | Contrato: equipos que participan, para revisar choques. | Programable |
| 6 | String | getDescripcion() | public abstract | Contrato: texto para el calendario. | Programable |
| 7 | int | compareTo(Programable otro) | public default | Orden del calendario; le delega a getHorario().compareTo(...). | Programable |
| 8 | void | registrarResultado(int marcadorLocal, int marcadorVisitante) | public abstract | Contrato: registra el marcador y reparte puntos de tabla. | Puntuable |
| 9 | boolean | esMarcadorValido(int marcadorLocal, int marcadorVisitante) | public abstract | Contrato: regla del marcador del deporte. | Puntuable |
| 10 | int | calcularPuntosTabla(int marcadorPropio, int marcadorRival) | public abstract | Contrato: puntos de tabla para un equipo. | Puntuable |
| 11 | boolean | estaFinalizado() | public abstract | Contrato: indica si ya tiene resultado. | Puntuable |
| 12 | String | getDescripcion() | public abstract | Contrato: texto para listar el encuentro. | Puntuable |

| | | | | | |
|---|---|---|---|---|---|
| 13 | String | toString() | public | @Override: devuelve el texto del estado. | EstadoEvento |
| 14 | — | Equipo(String codigo, String nombre, String empresa) | public | Constructor: valida textos no vacíos; puntos en 0. | Equipo |
| 15 | String / String / String / int | getCodigo(), getNombre(), getEmpresa(), getPuntosTabla() | public | Lecturas para la tabla y las búsquedas. | Equipo |
| 16 | void | sumarPuntos(int puntos) | public | Suma puntos de tabla (≥ 0); solo lo llama un encuentro al registrar su resultado. | Equipo |
| 17 | int | compareTo(Equipo otro) | public | @Override de Comparable: Integer.compare de los puntos al revés (mayor primero) y luego el nombre. | Equipo |
| 18 | boolean / int | equals(Object otro), hashCode() | public | @Override: dos equipos son iguales si tienen el mismo código. | Equipo |
| 19 | String | toString() | public | @Override: código, nombre, empresa y puntos. | Equipo |
| 20 | — | Horario(LocalDate fecha, int hora, String sede) | public | Constructor: valida fecha, hora de 0 a 23 y sede no vacía. | Horario |
| 21 | LocalDate / int / String | getFecha(), getHora(), getSede() | public | Lecturas del horario. | Horario |
| 22 | boolean | mismoMomento(Horario otro) | public | true si es la misma fecha y hora; sirve para el choque de equipos. | Horario |
| 23 | boolean | mismaSedeYMomento( Horario otro) | public | true si además coincide la sede; sirve para el choque de sede. | Horario |
| 24 | int | compareTo(Horario otro) | public | @Override de Comparable: fecha, luego hora y luego sede. Único criterio de orden. | Horario |
| 25 | String | toString() | public | @Override: "2026-11-12 09:00 - Gimnasio Norte". | Horario |
| 26 | — | Encuentro(Equipo local, Equipo visitante) | protected | Constructor que usan las subclases con super: valida equipos distintos, marcador 0-0, sin programar. | Encuentro |
| 27 | Equipo / Equipo | getLocal(), getVisitante() | public | Lecturas de los equipos. | Encuentro |
| 28 | int / int / EstadoEvento | getMarcadorLocal(), getMarcadorVisitante(), getEstado() | public | Lecturas del marcador y del estado. | Encuentro |
| 29 | String | getDeporte() | public abstract | Nombre del deporte; lo define cada subclase. | Encuentro |
| 30 | boolean | puedeProgramarse() | public | @Override de Programable: false si ya está finalizado. | Encuentro |
| 31 | void | programar(Horario horario) | public | @Override de Programable: lanza IllegalStateException si está finalizado; si no, guarda el horario y pasa a Programado. | Encuentro |
| 32 | boolean / Horario | estaProgramado(), getHorario() | public | @Override de Programable. | Encuentro |
| 33 | ArrayList<Equipo> | getParticipantes() | public | @Override de Programable: lista nueva con sus dos equipos. | Encuentro |

| | | | | | |
|---|---|---|---|---|---|
| 34 | String | getDescripcion() | public | @Override de Programable y Puntuable: "Fútbol sala: Faro vs Sol". | Encuentro |
| 35 | void | registrarResultado(int marcadorLocal, int marcadorVisitante) | public final | @Override de Puntuable: revisa estado, negativos y esMarcadorValido; luego guarda, finaliza y suma calcularPuntosTabla a cada equipo. | Encuentro |
| 36 | boolean | esMarcadorValido(int marcadorLocal, int marcadorVisitante) | public abstract | Queda sin resolver; lo resuelve cada deporte. | Encuentro |
| 37 | int | calcularPuntosTabla(int marcadorPropio, int marcadorRival) | public abstract | Queda sin resolver; lo resuelve cada deporte. | Encuentro |
| 38 | boolean | estaFinalizado() | public | @Override de Puntuable. | Encuentro |
| 39 | String | toString() | public | @Override: descripción, estado, horario y resultado si lo tiene. | Encuentro |
| 40 | — | FutbolSala(Equipo local, Equipo visitante) | public | Constructor: llama a super( local, visitante). | FutbolSala |
| 41 | String | getDeporte() | public | @Override: devuelve "Fútbol sala". | FutbolSala |
| 42 | boolean | esMarcadorValido(int marcadorLocal, int marcadorVisitante) | public | @Override: siempre true; el empate se permite. | FutbolSala |
| 43 | int | calcularPuntosTabla(int marcadorPropio, int marcadorRival) | public | @Override: 3 si gana, 1 si empata y 0 si pierde. | FutbolSala |
| 44 | — | Baloncesto(Equipo local, Equipo visitante) | public | Constructor: llama a super( local, visitante). | Baloncesto |
| 45 | String | getDeporte() | public | @Override: devuelve "Baloncesto". | Baloncesto |
| 46 | boolean | esMarcadorValido(int marcadorLocal, int marcadorVisitante) | public | @Override: false si el marcador es igual (no hay empate). | Baloncesto |
| 47 | int | calcularPuntosTabla(int marcadorPropio, int marcadorRival) | public | @Override: 2 si gana y 1 si pierde. | Baloncesto |
| 48 | — | Exhibicion(String titulo) | public | Constructor: valida título no vacío; queda sin programar. | Exhibicion |
| 49 | String / EstadoEvento | getTitulo(), getEstado() | public | Lecturas. | Exhibicion |
| 50 | boolean | puedeProgramarse() | public | @Override de Programable: siempre true. | Exhibicion |
| 51 | void | programar(Horario horario) | public | @Override de Programable: guarda el horario y pasa a Programado. | Exhibicion |
| 52 | boolean / Horario | estaProgramado(), getHorario() | public | @Override de Programable. | Exhibicion |
| 53 | ArrayList<Equipo> | getParticipantes() | public | @Override de Programable: lista vacía (no tiene equipos). | Exhibicion |
| 54 | String | getDescripcion() | public | @Override de Programable: "Exhibición: Clínica de baloncesto". | Exhibicion |
| 55 | String | toString() | public | @Override: descripción, estado y horario. | Exhibicion |

| | | | | | |
|---|---|---|---|---|---|
| 56 | — | Copa() | public | Constructor: las tres listas vacías. | Copa |
| 57 | void | registrarEquipo(Equipo equipo) | public | Agrega el equipo; lanza IllegalArgumentException si el código ya existe. | Copa |
| 58 | void | registrarEquipo(String codigo, String nombre, String empresa) | public | Overloading: crea el equipo y llama a la versión anterior. | Copa |
| 59 | Equipo | buscarEquipo(String codigo) | public | Devuelve el equipo o null. | Copa |
| 60 | void | agregarEvento(Programable evento) | public | Agrega un evento que solo va al calendario (exhibición). | Copa |
| 61 | void | agregarEncuentro(Encuentro encuentro) | public | Lo agrega a eventos y a encuentros. | Copa |
| 62 | ArrayList<Equipo> / ArrayList< Programable> / ArrayList<Puntuable> | getEquipos(), getEventos(), getEncuentros() | public | Copias de las listas para mostrarlas sin exponer las internas. | Copa |
| 63 | void | programarEvento(int numeroEvento, LocalDate fecha, int hora, String sede) | public | Crea el Horario, revisa que se pueda programar, la sede y los equipos, y solo después llama a programar. | Copa |
| 64 | void | registrarResultado(int numeroEncuentro, int marcadorLocal, int marcadorVisitante) | public | Busca el Puntuable y le delega el registro. | Copa |
| 65 | ArrayList<Equipo> | getTablaPosiciones() | public | Copia de los equipos ordenada con Collections.sort (usa compareTo de Equipo). | Copa |
| 66 | Iterator< Programable> | iterator() | public | @Override de Iterable: copia de los eventos programados, ordenada con Collections.sort. | Copa |
| 67 | Programable / Puntuable | obtenerEvento(int numero), obtenerEncuentro(int numero) | private | Devuelven el elemento o lanzan excepción si el número no existe. | Copa |
| 68 | boolean | sedeOcupada(Programable evento, Horario horario) | private | true si otro evento usa esa sede a esa fecha y hora. | Copa |
| 69 | Equipo | equipoOcupado(Programable evento, Horario horario) | private | Devuelve el equipo que ya juega a esa hora o null; usa getParticipantes(). | Copa |
| 70 | void | main(String[] args) | public static | Carga los datos y muestra el menú en un do-while hasta elegir 0. | Main |
| 71 | void | cargarDatosIniciales() | private static | 4 equipos, 7 encuentros (5 con resultado) y 2 exhibiciones, con los mismos métodos de Copa. | Main |
| 72 | void | mostrarMenu() | private static | Imprime las opciones. | Main |
| 73 | void | registrarEquipo() / crearExhibicion() | private static | Piden los datos y muestran el resultado o el error. | Main |
| 74 | void | crearEncuentro() | private static | Pide equipos y deporte; único lugar que crea FutbolSala o Baloncesto. | Main |

| | | | | | |
|---|---|---|---|---|---|
| 75 | void | programarEvento() / registrarResultado() | private static | Muestran la lista, piden los datos y atrapan IllegalArgumentException e IllegalStateException. | Main |
| 76 | void | mostrarTablaPosiciones() | private static | Imprime getTablaPosiciones(). | Main |
| 77 | void | mostrarCalendario() | private static | Recorre la copa con for-each (Iterable). | Main |
| 78 | void | mostrarEventos() / mostrarEquipos() | private static | Listan los eventos numerados y los equipos. | Main |
| 79 | String / int / LocalDate | leerTexto(String), leerEntero( String), leerFecha(String) | private static | Lectura segura: repiten la pregunta si el formato es incorrecto. | Main |

Cada objeto recibe sus datos por el constructor. Si algún dato no sirve por ejemplo un texto vacío, una hora fuera de rango o dos equipos iguales, el constructor lo rechaza, así nunca se crea un objeto a medias. Un equipo empieza con 0 puntos y una exhibición empieza sin programar.

En los encuentros los constructores trabajan en cadena. Al crear un FutbolSala o un Baloncesto, su constructor llama a super, que es el constructor de Encuentro. Ese constructor revisa que los equipos sean distintos, deja el marcador en 0-0 y el estado en sin programar. Las subclases no tienen más datos que guardar, solo sus constantes de puntos. La copa empieza vacía, y los datos iniciales se cargan con los mismos métodos que usa el menú, así pasan por las mismas revisiones.

## 5. Por qué dos interfaces y no una

El calendario solo necesita saber si algo se puede programar y cuándo es. La tabla necesita algo distinto: que reciba un resultado y reparta puntos. Los encuentros hacen las dos cosas, pero las exhibiciones solo la primera. Por eso usamos dos interfaces: cada clase cumple solo lo que de verdad puede hacer.

Si las exhibiciones tuvieran que cumplir la interfaz de la tabla, tendrían que tener métodos para registrar resultados y calcular puntos aunque no tienen equipos ni marcador. Esos métodos quedarían vacíos o lanzando un error. La tabla terminaría con eventos que dicen que dan puntos pero no lo hacen y, para evitar fallas, habría que preguntar a cada rato si el evento es una exhibición, que es justo lo que queremos evitar.

## 6. Agregar un tercer deporte

Para agregar, por ejemplo, voleibol, solo hay que crear una clase Voleibol que herede de Encuentro y escribir sus tres reglas: el nombre del deporte, cuándo un marcador es válido y cuántos puntos da. En el menú se agrega la opción para crear un encuentro de voleibol.

La tabla de posiciones y el calendario no cambian. Los dos trabajan con las interfaces y no con los deportes, así que el voleibol funciona con ellos desde el primer momento. Tampoco hay que tocar Encuentro, Equipo, Horario, Exhibicion ni Copa.